

编译原理-中间代码生成 03

控制流的 SDT 方案及编译优化选讲

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 5 月 22 日



福州大学

FUZHOU UNIVERSITY



- ① 知识点回顾
- ② 控制结构与布尔表达式
- ③ 控制流回填
- ④ switch...case 语句的翻译
- ⑤ 静态单赋值形式 (SSA)
- ⑥ 优化方法示例

- 中间表示的分类
- 三地址码
- 中间代码生成策略
- 基于类型表达式的声明处理
- 表达式翻译

⑤ 静态单赋值形式 (SSA)

表达式翻译

⑤ 静态单赋值形式 (SSA)

H/M/LIR 示例

源代码

```
float a[10][20];  
a[i][j+2];
```

HIR

```
t1=a[i,j+2];
```

MIR

```
t1=j+2;  
t2=i*20;  
t3=t1+t2;  
t4=4*t3;  
t5=addr a;  
t6=t5+t4;  
t7=*t6;
```

LIR

```
r1=[fp-4];
r2=r1+2;
r3=[fp-8];
r4=r3*20;
r5=r4+t2;
r6=4*r5;
r7=fp-816;
f1=[r7+r6];
```

- 逆波兰式（后缀式）
- 图结构表示
- 三地址码

表达式翻译

⑤ 静态单赋值形式 (SSA)

9 / 61

常见三地址码	翻译为 x86 汇编	翻译为 ARM 汇编
<code>x = y</code>	<code>mov eax, [b] ; 将 b 的值加载到寄存器 eax</code> <code>mov [a], eax ; 将 eax 的值存入 a</code>	<code>LDR r0, [b] ; 从内存加载 b 的值到寄存器 r0</code> <code>STR r0, [a] ; 将 r0 的值存入内存地址 a</code>
<code>x = y + z</code>	<code>mov eax, [b] ; 加载 b 到 eax</code> <code>add eax, [c] ; eax = eax + c</code> <code>mov [a], eax ; 结果存入 a</code>	<code>LDR r0, [b] ; 加载 b 到 r0</code> <code>LDR r1, [c] ; 加载 c 到 r1</code> <code>ADD r0, r0, r1 ; r0 = r0 + r1</code> <code>STR r0, [a] ; 结果存入 a</code>
<code>if a < b goto L</code>	<code>mov eax, [a]</code> <code>cmp eax, [b] ; 比较 a 和 b</code> <code>jlt L ; 若 a < b (有符号比较), 跳转到标签 L</code>	<code>LDR r0, [a]</code> <code>LDR r1, [b]</code> <code>CMP r0, r1 ; 比较 r0 和 r1, 设置标志位</code> <code>BLT L ; 若 r0 < r1 (有符号), 跳转到标签 L</code>
<code>goto L</code>	<code>jmp L ; 无条件跳转到标签 L</code>	<code>B L ; 无条件跳转到标签 L</code>
<code>a = b[i];¹</code>	<code>mov esi, [i] ; 加载索引 i 到 esi</code> <code>mov eax, [b + esi*4] ; 计算地址 b + i*4, 读取值到 eax</code> <code>mov [a], eax</code>	<code>LDR r0, =b ; 加载数组基址到 r0</code> <code>LDR r1, [i] ; 加载索引 i 到 r1</code> <code>LDR r0, [r0, r1, LSL #2] ; 地址计算: 基址 + (i << 2) (即 i*4)</code> <code>STR r0, [a] ; 结果存入 a</code>
<code>b[i] = a</code>	<code>mov esi, [i]</code> <code>mov eax, [a]</code> <code>mov [b + esi*4], eax ; 将 a 的值写入 b[i]</code>	<code>LDR r0, =b</code> <code>LDR r1, [i]</code> <code>LDR r2, [a]</code> <code>STR r2, [r0, r1, LSL #2] ; 将 a 的值写入 b[i]</code>

¹int b[10];

三地址码的表示方法

- 三地址码说明了各类指令的组成部分，但未描述指令在**某个数据结构中的表示**方法。它可以用**四元式**、**三元式**和**间接三元式**来描述。
- **四元式**是一个带有 4 个字段的记录结构：op, arg1, arg2, result, 其中 arg1, arg2 和 result 通常指向符号表项目的入口。
- **三元式**是 3 个字段的记录结构：op, arg1, arg2:
 - 三元式可以避免引入临时变量（如 t1, t2, t3）；
 - 使用运算的位置来引用它的运算结果；
 - 用带有括号的数字表示指向三元式结构的指针；
- 当移动一个变量的指令时，使用三元式表示法便需要修改引用该指令地址的所有指令。——使用**间接三元式**予以解决：
 - 间接三元式方案包括三元式表 + 间接码表；
 - 间接码表（指针数组）：一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置，优化编译器可以通过对该表的重排来移动指令的位置。

综合示例

例

构造下述语句的三地址码、四元式、三元式、间接三元式：

$X = (A + B) * C;$

$Y = D \uparrow (A + B);$

四元式表示法：

三元式表示法：

间接码表：

三地址码：

$t1 = A + B;$

$X = t1 * C;$

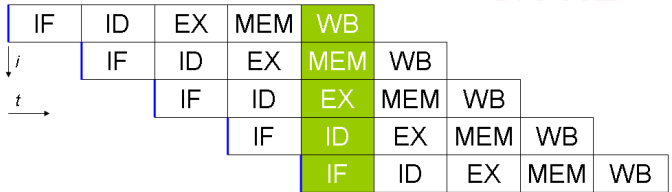
$Y = D \uparrow t1;$

序号	op	arg1	arg2	result
(0)	+	A	B	t1
(1)	*	C	t1	t2
(2)	=	t2	N/A	X
(3)	↑	D	t1	t3
(4)	=	t3	N/A	Y

序号	op	arg1	arg2
(0)	+	A	B
(1)	*	C	(0)
(2)	=	X	(1)
(3)	↑	D	(0)
(4)	=	Y	(3)

序号	Statement No.
(10)	(0)
(11)	(1)
(12)	(3)
(13)	(2)
(14)	(4)

重排指令顺序的必要性



RISC 机器经典五级流水示意图

Pipeline Stage	Clock Cycle					
	1	2	3	4	5	6
Fetch	SUB	AND				
Decode		SUB	AND			
Execute			SUB	AND		
Access				SUB	AND	
Write-Back					SUB	AND

指令级并行时的数据冒险



① 知识点回顾

中间表示的分类

三地址码

中间代码生成策略

基于类型表达式的声明处理

表达式翻译

② 控制结构与布尔表达式

③ 控制流回填

④ switch...case 语句的翻译

⑤ 静态单赋值形式 (SSA)

中间代码生成策略

- 生成流程：
 - 输入：经过**语法分析与语义分析**的**抽象语法树或符号表**。
 - 输出：与**目标机器无关**的**三地址码、四元式或其他中间形式**。
 - 关键步骤：
 - 声明处理：将变量类型、作用域等信息登记至符号表，分配存储地址（如相对偏移量）。
 - **表达式翻译**：递归遍历语法树，生成运算序列（如利用语法制导翻译生成四元式）。
 - **控制结构处理**：为条件语句、循环语句生成跳转指令标签。
- 优化技术：
 - 常量传播：替换编译时可确定的常量值。
 - **死代码消除**：删除不可达或无效代码段。
 - **公共子表达式提取**：重用重复计算的中间结果。



① 知识点回顾

中间表示的分类

三地址码

中间代码生成策略

基于类型表达式的声明处理

表达式翻译

② 控制结构与布尔表达式

③ 控制流回填

④ switch...case 语句的翻译

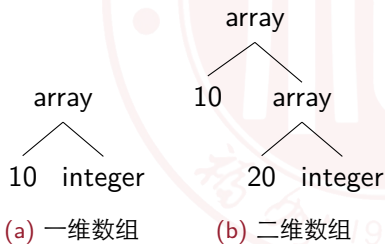
⑤ 静态单赋值形式 (SSA)

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - ① 确定一个名需要的存储空间；
 - ② 计算一个数组元素引用的地址；
 - ③ 插入显式的类型转换；
 - ④ 选择算术运算符的正确版本；

常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;
 - 类型别名与一串类型表达式等价;
- 数组类型: 一维数组、多维数组; 如 `array(N, T)` 说明了一个有 `N` 个 `T` 类型元素构成的数组。



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;
 - 类型别名与一串类型表达式等价;
- 数组类型: 一维数组、多维数组; 如 `array(N, T)` 说明了一个有 10 个 T 类型元素构成的数组类型。
- 指针类型: 用 `pointer(T)` 表示指向数据类型 T 的指针类型。
- 函数声明符 $\rightarrow$: 若 T_1 、 T_2 、 $\dots$ 、 T_n 和 R 是类型表达式, 则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是合法的类型表达式, 用于表示一个函数。
- **记录**²构造符 `record`: 若有标识符 N_1 、 N_2 、 $\dots$ 、 N_n 与类型表达式 T_1 、 T_2 、 $\dots$ 、 T_n , 则 `record(( $N_1 \times T_1$ )  $\times$  ( $N_2 \times T_2$ )  $\times \dots \times$  ( $N_n \times T_n$ ))` 是类型表达式。

²结构体、类、命名元组 (namedtuple)

声明语句的翻译

- 对于声明语句，语义分析的主要任务是收集标识符的类型、所需存储单元大小等属性信息，并为每个名字分配一个相对地址，并将它们保存在相应的符号表记录中。
 - 从类型表达式可以知道该类型 (type) 在运行时刻所需的存储单元数量称为类型的宽度 (width)
 - 在编译时刻，可以使用类型的宽度为每一个名字分配一个相对地址 (offset)

变量声明语句的 SDD

变量声明的基础文法：

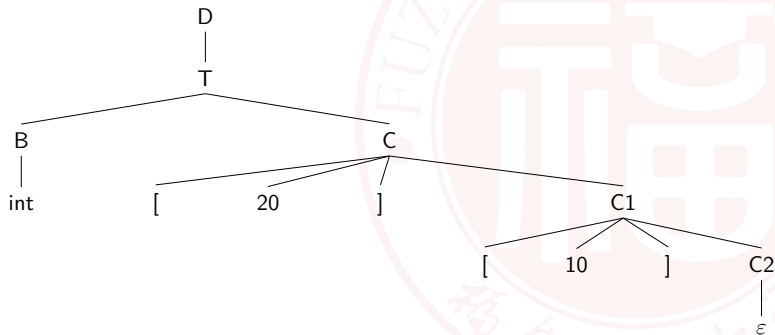
- ① $P \rightarrow D$
- ② $D \rightarrow T \text{ id}; D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow BC$
- ⑤ $B \rightarrow \text{int}$
- ⑥ $B \rightarrow \text{real}$
- ⑦ $C \rightarrow \varepsilon$
- ⑧ $C \rightarrow [\text{num}]C1$

- 翻译要求：确定每个变量的**类型**表达式和相对地址。
- SDD 属性设计：
 - B, C, T 的综合属性 type, width;
- SDT 方案需要的辅助变量：
 - offset：下一个可用的相对地址；
- 在处理第一个声明之前，offset 被设置为 0。每处理一个变量 x，x 被加入符号表，它的相对地址被设置为 offset 的当前值，其宽度被加到 offset 上。
- 类型表达式的生成需要将类型和宽度信息从语法分析树中的 B 结点传递到对应于产生式 $C \rightarrow \varepsilon$ 的结点，可设置继承属性 inh 实现。此外，考虑到解析类型过程是**连续**的，可以使用临时变量 t 和 w 暂存 B 的 type 和 width 值。

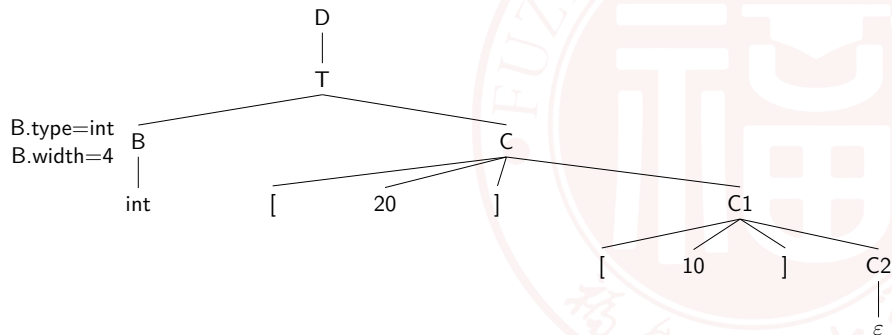
变量声明语句的 SDT

- ① $P \rightarrow \{\text{offset} = 0;\} D$
- ② $D \rightarrow T \text{ id}; \{\text{st}[\text{top}].\text{put}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width};\} D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B \{ t = B.\text{type}; w = B.\text{width}; \} C \{ T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width}; \}$
- ⑤ $B \rightarrow \text{int} \{ B.\text{type} = \text{int}; B.\text{width} = 4; \}$
- ⑥ $B \rightarrow \text{real} \{ B.\text{type} = \text{real}; B.\text{width} = 8; \}$
- ⑦ $C \rightarrow \varepsilon \{ C.\text{type} = t; C.\text{width} = w; \}$
- ⑧ $C \rightarrow [\text{num}]C1 \{ C.\text{type} = \text{array}(\text{num.val}, C1.\text{type}); \\ C.\text{width} = \text{num.val} * C1.\text{width}; \}$

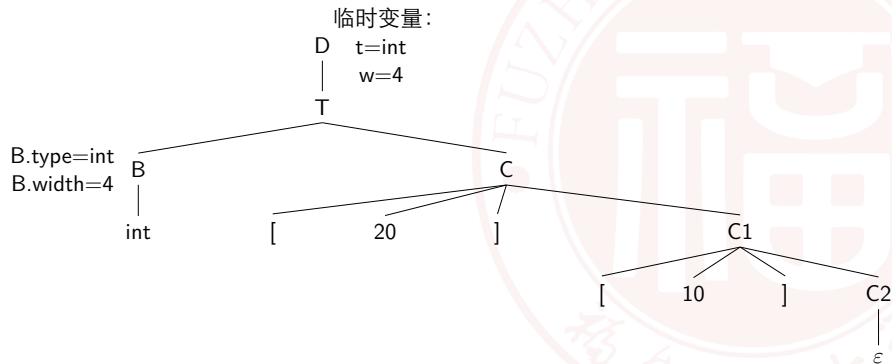
例 (变量声明的注释分析树)



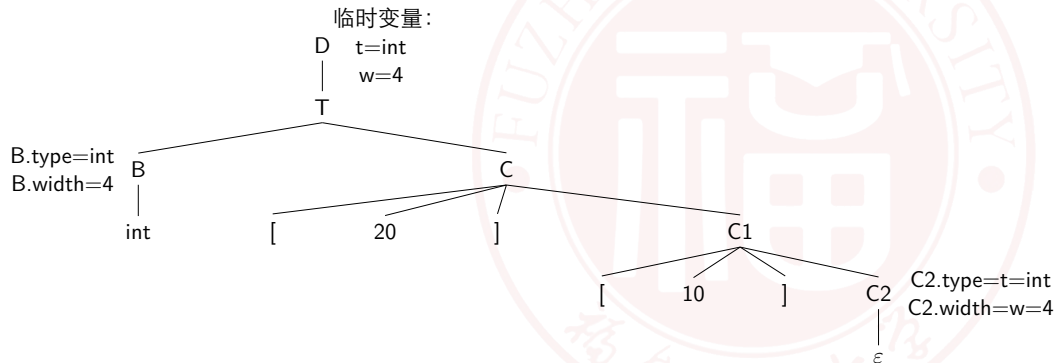
例 (变量声明的注释分析树)



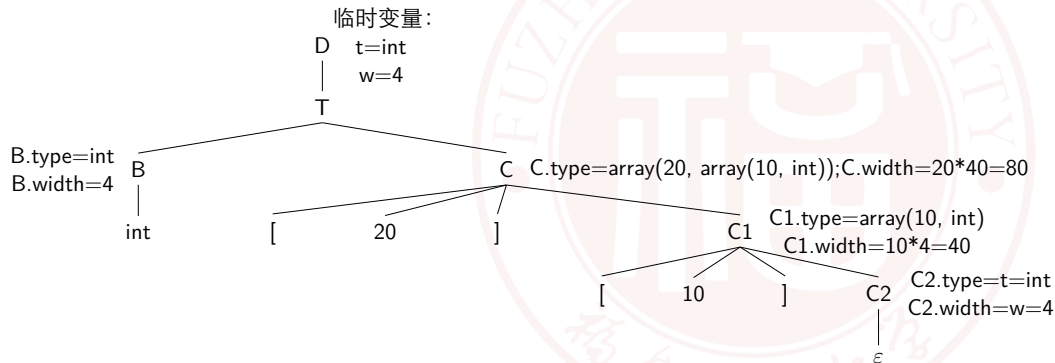
例 (变量声明的注释分析树)



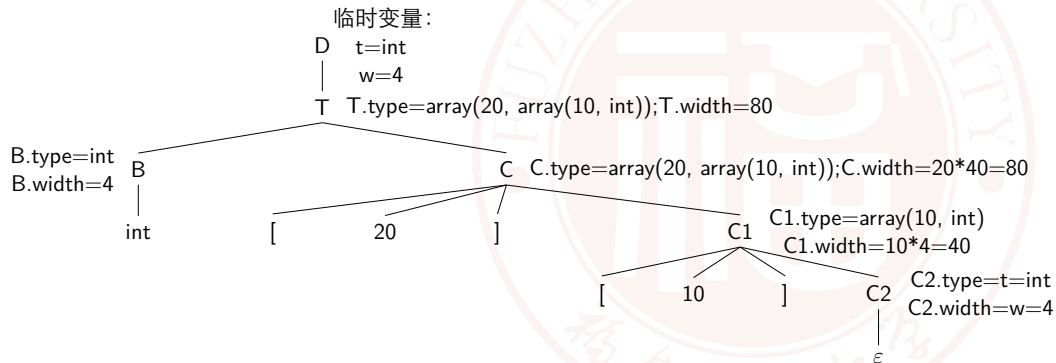
例 (变量声明的注释分析树)



例 (变量声明的注释分析树)



例 (变量声明的注释分析树)





① 知识点回顾

中间表示的分类

三地址码

中间代码生成策略

基于类型表达式的声明处理

表达式翻译

② 控制结构与布尔表达式

③ 控制流回填

④ switch...case 语句的翻译

⑤ 静态单赋值形式 (SSA)

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 TAC 序列。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 **TAC 序列**。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:
 - 综合属性 code: 赋值语句 S 的三地址代码。

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 **TAC 序列**。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:
 - 综合属性 code: 赋值语句 S 的三地址代码。
- 对于 E 符号:

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 TAC 序列。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:
 - 综合属性 code: 赋值语句 S 的三地址代码。
- 对于 E 符号:
 - 综合属性 code: 对 E 求值的三地址语句序列;

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 **TAC 序列**。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:
 - 综合属性 code: 赋值语句 S 的三地址代码。
- 对于 E 符号:
 - 综合属性 code: 对 E 求值的三地址语句序列;
 - 综合属性 addr: 存放 E 值的地址 (名字或临时变量);

简单算术表达式及赋值语句

基本文法:

- ① $S \rightarrow id = E;$
- ② $E \rightarrow E1 + E2$
- ③ $E \rightarrow E1 * E2$
- ④ $E \rightarrow -E1$
- ⑤ $E \rightarrow (E1)$
- ⑥ $E \rightarrow id$

目标: 生成对表达式求值的 TAC 序列。

例

```
x=(a+b)*c
⇒
t1 = a + b;
t2 = t1 * c;
x = t2;
```

属性文法:

- 对于 S 符号:
 - 综合属性 code: 赋值语句 S 的三地址代码。
- 对于 E 符号:
 - 综合属性 code: 对 E 求值的三地址语句序列;
 - 综合属性 addr: 存放 E 值的地址 (名字或临时变量);
- 全局辅助函数 newTemp(): 生成新的临时变量 t, 返回 t 的地址 (名字);

简单算术表达式及赋值语句

赋值语句 SDT:

- ① $S \rightarrow id = E; \{ S.code = E.code || gen(st.get(id.lexeme), "=", E.addr); \}$
- ② $E \rightarrow E1 + E2 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "+", E2.addr); \}$
- ③ $E \rightarrow E1 * E2 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", E1.addr, "*", E2.addr); \}$
- ④ $E \rightarrow -E1 \{ E.addr = newTemp(); E.code = E1.code || E2.code || gen(E.addr, "=", "uminus", E1.addr); \}$
- ⑤ $E \rightarrow (E1) \{ E.addr = E1.addr(); E.code = E1.code; \}$
- ⑥ $E \rightarrow id \{ E.addr = st.get(id.lexeme); E.code = ""; \}$

增量翻译

- E 的三地址码为左儿子的三地址码，连接右儿子的三地址码，再连接一条三地址码，
- 其儿子的三地址码为其儿子的儿子三地址码序列的连接...
- ...
- 代码序列可能很长，重复使用字符串拼接过程效率低。
- 可以采用**增量**的方式来生成三地址码。
- 在增量翻译中，`gen()` 不仅要构造出一个新的三地址指令，还要将它添加到至今为止已生成的指令序列之后。

基于增量翻译的 SDT 方案

- ① $S \rightarrow id = E; \{ \text{gen}(\text{st.get}(id.\text{lexeme}), "=", E.\text{addr}); \}$
- ② $E \rightarrow E1 + E2 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", E1.\text{addr}, "+", E2.\text{addr}); \}$
- ③ $E \rightarrow E1 * E2 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", E1.\text{addr}, "*", E2.\text{addr}); \}$
- ④ $E \rightarrow -E1 \{ E.\text{addr} = \text{newTemp}(); \text{gen}(E.\text{addr}, "=", "uminus", E1.\text{addr}); \}$
- ⑤ $E \rightarrow (E1) \{ E.\text{addr} = E1.\text{addr}(); \}$
- ⑥ $E \rightarrow id \{ E.\text{addr} = \text{st.get}(id.\text{lexeme}); \}$

数组引用寻址的 SDT

- 数组引用翻译方案的设计中，**关键的问题**是如何将地址计算公式同数组引用的文法关联起来。
- 设非终结符 L 生成一个标识符连接一个下标表达式的序列：

$$L \rightarrow id[E] \mid L_1[E]$$

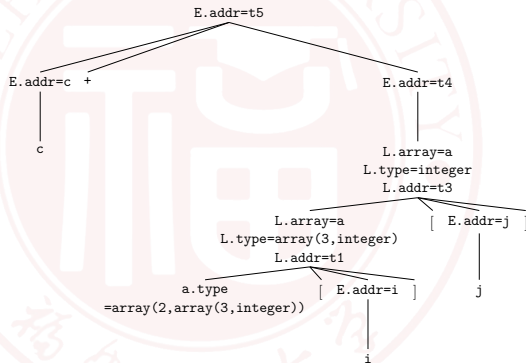
- L 的综合属性：
 - $L.addr$ 指示一个临时变量，表示 $\sum_j i_j * w_j$;
 - $L.array$ 是指向数组名字对应符号表条目的指针，其基地址为 $L.array.base$;
 - $L.type$ 是 L 生成的子数组的类型。对于任意数组类型 t ，假定 $t.width$ 给出它的宽度， $t.elem$ 给出其元素的类型。

数组 `int a[2][3]`; 和表达式 `c+a[i][j]`

```

S → id=E;      { gen(top.get(id.lexeme), '=', E.addr); }
  | L=E;        { gen(L.array.base, '[' , L.addr, ']', '=', E.addr); }
E → E1 + E2    { E.addr=newTemp();
                { gen(E.addr, '=', E1.addr, '+', E2.addr); }
  | id          { E.addr=top.get(id.lexeme); }
  | L           { E.addr=newTemp();
                { gen(E.addr, '=', L.array.base, '[' , L.addr, ']', ); }
L → id [ E ]   { L.array=top.get(id.lexeme);
                L.type=L.array.type.elem;
                L.addr=newTemp();
                { gen(L.addr, '=', E.addr, '*', L.type.width); }
  | L1 [ E ]   { L.array=L1.array;
                L.type=L1.type.elem;
                t=newTemp();
                L.addr=newTemp();
                { gen(t, '=', E.addr, '*', L.type.width);
                  gen(L.addr, '=', L1.addr, '+', t); }

```



(20, array(10, int)); T.width=80

C C.type=array(20, array(10, int)); C.width=20*40=80

] C1 C1.type=array(10, int)
C1.width=10*4=40

[10] C2 C2.type=t=ir
C2.width=w=

ϵ

类型检查及类型转换

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行一些自动的类型转换；
- 拓宽转换和窄化转换：
 - 拓宽：较低层的类型转换为较高层，即转化为表达能力更好的数据类型
 - `short`→`int`→`float`→`double`；
 - `unsigned int`→`int`；
 - 窄化：较高层到较低层，如派生类转换为基类。
- 隐式转换和显式转换：
 - 隐式：由编译器自动完成（隐式类型转化链）；
 - 显式（强制类型转换）：程序员写出代码；
- 添加必要的类型转换代码，如函数调用等。

- ① 知识点回顾
- ② 控制结构与布尔表达式
控制流语句的代码结构
- ③ 控制流回填
- ④ switch...case 语句的翻译
- ⑤ 静态单赋值形式 (SSA)
- ⑥ 优化方法示例

if-then-else 和 while-do 等) 中的条件表达式。
 式的翻译是**结合**在一起的。
 指令构成的**跳转代码序列**。
 尔变量或关系表达式结合在一起。

- 布尔表达式的作用：
 - 用作逻辑演算，计算逻辑值；
 - 用作控制流语句（如 if-then, if-then-else 和 while-do 等）中的条件表达式。
- 控制流语句的翻译和对布尔表达式的翻译是**结合**在一起的。
- 布尔表达式 B 被翻译成**由跳转指令构成的跳转代码序列**。
- 布尔表达式通过布尔运算符将布尔变量或关系表达式结合在一起。

布尔表达式值的计算方法

数值法:

- 用数值表示真和假 (1 表示真, 0 表示假), 对布尔表达式的求值可以象对算术表达式的求值那样一步一步地来计算。

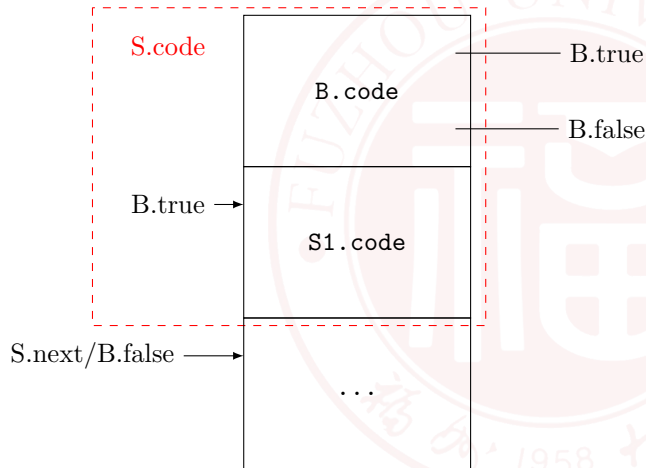
例

```
1||(!0&&0)||0
=1||(1&&0)||0
=1||0||0
=1||0
=1
```

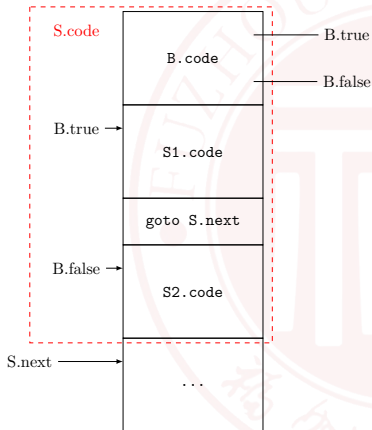
短路跳转法:

- 根据布尔表达式的特点:
 - A || B: 若 A 为真, 则 B 的值就不必计算, A || B 的值为真。
 - A && B: 若 A 为假, 则 B 的值不必计算, A && B 的值为假。

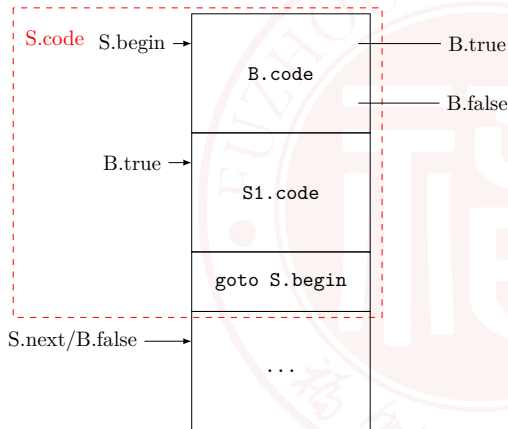
$S \rightarrow \text{if } (B) \text{ then } S1;$



$S \rightarrow \text{if } (B) \text{ then } S1 \text{ else } S2;$



$S \rightarrow \text{while } (B) \text{ do } S1;$



控制流语句的 SDT

- **关键：用指令的标号标识一条三地址指令。**
- 属性文法：
 - 对于符号 B、S 有综合属性 code，表示它们翻译后的三地址码；
 - 对于符号 S 有继承属性 next，存放紧跟在 S 代码之后的指令的标号；
 - 对于符号 B 有继承属性：
 - true，存放当 B 为真时控制流转向的指令的标号；
 - false，是一个地址，存放当 B 为假时控制流转向的指令的标号；

顺序结构的 SDT

- $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$
- $S \rightarrow \{ S1.next = newlabel(); \} S1 \{ label(S1.next); S2.next = S.next ; \} S2$
- $S \rightarrow id = E ; \mid L = E ;$
- $S \rightarrow if (B) S1 \mid if (B) S1 else S2 \mid while (B) do S1$

其中,

- newlabel() 用于产生一个新的标号;
- label(L) 用于将标签 L 赋予下一个调用 gen 函数生成的三地址码;
 - 特别地, 我们可以假设存在一个变量 statCnt 用于计数已经生成了多少条 TAC, 则 label(L) 可以认为执行的是 $L = statCnt$;

```

S2.next = S.next; } B { label(B.true); S1.next =
S2.next; } S1 { label(S1.true); S2.next =
newlabel(); } B { label(B.true); S1.next =
S2.next; } else { label(B.false); S2.next = S.next; } S2
.next = S.next; } S1 { label(S1.true); S2.next =
label(S.begin); B.true = newlabel(); B.false =
newlabel(); S2.next = S2.next; } S1 { label(S1.true);
S2.next = S2.next; } S1 { gen('goto', S.begin); }

```

$S \rightarrow \text{while } (\{ S.\text{begin} = \text{newlabel}(); \text{label}(S.\text{begin}); B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \} B \{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{begin}; \}) S1 \{ \text{gen}('goto', S.\text{begin}); \}$

分支语句的 SDT

if (B) S1 的翻译方案

$S \rightarrow \text{if} (\{ B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \} B \{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{next}; \}) S1$

if (B) S1 else S2 的翻译方案

$S \rightarrow \text{if} (\{ B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}(); \} B \{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{next}; \}) S1 \{ \text{gen}('goto', S.\text{next}) \} \text{else} \{ \text{label}(B.\text{false}); S2.\text{next} = S.\text{next}; \} S2$

while (B) S1 的翻译方案

$S \rightarrow \text{while} (\{ S.\text{begin} = \text{newlabel}(); \text{label}(S.\text{begin}); B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next}; \} B \{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{begin}; \}) S1 \{ \text{gen}('goto', S.\text{begin}); \}$

这个 SDT 方案有没有什么问题?

布尔表达式的 SDT

```

B  →  E1 relop E2 { gen('if', E1.addr, rel.op, E2.addr, 'goto', B.true); gen('goto', B.false); }
      |  true { gen('goto', B.true); }
      |  false { gen('goto', B.false); }
      |  ( { B1.true = B.true; B1.false = B.false; } B1 )
      |  !{ B1.true = B.false; B1.false = B.true; } B1
      |  { B1.true = B.true; B1.false = newlabel(); } B1 || { label(B1.false); B2.true = B.true; B2.false = B.false; } B2
      |  { B1.true = newlabel(); B1.false = B.false; } B1 && { label(B1.true); B2.true = B.true; B2.false = B.false; } B2

```

例题

例 (if(x<100 || x >200 && x!=y) x=0; 的三地址码)

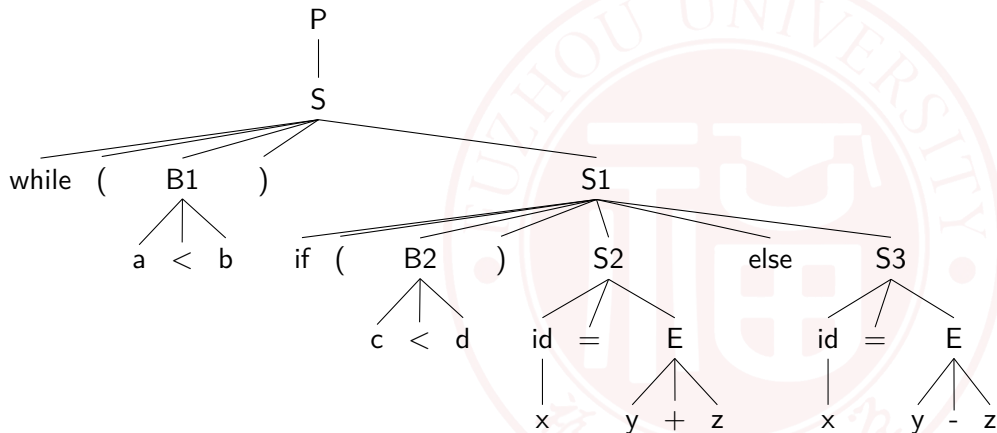
```
        if x<100 goto L2
        goto L3
L3:      if x>200 goto L4
        goto L1
L4:      if x!=y goto L2
        goto L1
L2:      x=0
L1:
```

例题 2

例 (写出 while (a<b) if (c<d) x=y+z; else x=y-z; 的三地址码)

相关 SDT 如下:

- $P \rightarrow \{ S.next = newlabel(); \} S \{ label(S.next); \}$
- $S \rightarrow while (\{ S.begin = newlabel(); label(S.begin); B.true = newlabel(); B.false = S.next; \} B \{ label(B.true); S1.next = S.begin; \}) S1 \{ gen('goto', S.begin); \}$
- $S \rightarrow if (\{ B.true = newlabel(); B.false = newlabel(); \} B \{ label(B.true); S1.next = S.next; \}) S1 \{ gen('goto', S.next) \} else \{ label(B.false); S2.next = S.next; \} S2$
- $S \rightarrow id = E ; \mid L = E ;$





- ① 知识点回顾
- ② 控制结构与布尔表达式
- ③ 控制流回填
- ④ switch...case 语句的翻译
- ⑤ 静态单赋值形式 (SSA)
- ⑥ 优化方法示例

- **控制流语句**翻译的一个**关键**是确定**跳转指令**的目标**标号**；
- 存在问题：生成跳转指令时，目标标号还**不能确定**³，需要以后扫描到特定位置时才能**回过头**来确定。
- 解决办法：
 - 两遍扫描 (L 属性定义)：生成一些临时变量用来存放标号，将临时变量的地址作为继承属性传递到标号可以确定的地方。即当目标标号的值确定下来以后再赋给相应的变量。此时，需要再扫描一次指令序列。
 - 一遍扫描 (S 属性定义)：生成一个跳转指令时，暂时不指定它的目标标号。这样的指令都被放入由跳转指令组成的列表中。同一个列表中的所有跳转指令具有相同的目标标号。等到能够确定正确的目标标号时，才去填充这些指令的目标标号。
 - 采用综合属性 truelist 和 falselist 管理跳转指令中的标记。
 - 采用回填：在目标地址确定时填充这些标记
 - 使用数据结构和现代汇编指令：
 - 使用结构体表示 TAC，保留一个字段 Label 来表示该 TAC 是否有一个标记；
 - 现代汇编指令支持字符串标号，跳转指令也是支持的，所以没必要计算成精确的位置。

³其实标号是可以确定的，只是这个标号对应的是哪一条指令是不知道的。



- ① 知识点回顾
- ② 控制结构与布尔表达式
- ③ 控制流回填
- ④ switch...case 语句的翻译
- ⑤ 静态单赋值形式 (SSA)
- ⑥ 优化方法示例

- 在编译过程中, switch-case 结构通常会被翻译成高效的条件跳转逻辑, 而具体的三地址码 (Three-Address Code, TAC) 生成策略取决于 case 标签的分布 (密集或稀疏)。
- 跳转表 (Jump Table)⁴: 适用于 case 标签密集且连续的情况 (如 1,2,3,4), 编译器会生成一个直接通过索引跳转的代码块, 时间复杂度为 $O(1)$ 。
- 条件分支链 (Conditional Branch Chain): 适用于 case 标签稀疏的情况 (如 10, 20, 30), 编译器生成一系列 if-else 条件判断, 时间复杂度为 $O(n)$ 。
- 二分查找优化: 当 case 数量多且范围大时 (如 100, 200, 300, ..., 1000), 编译器可能生成二分查找逻辑, 将时间复杂度优化到 $O(\log n)$ 。
- 特别需要注意的地方:
 - Default 处理: 所有未匹配的 case 必须跳转到 default 分支。
 - Fall-through 消除: break 语句会被翻译为显式的 goto end_switch, 避免执行后续 case。
 - 值范围检查: 跳转表需先验证输入值是否在合法范围内 (防止越界访问)。

⁴此处体现了代码亦是数据思想



- ① 知识点回顾
- ② 控制结构与布尔表达式
- ③ 控制流回填
- ④ switch...case 语句的翻译
- ⑤ 静态单赋值形式 (SSA)
- ⑥ 优化方法示例

SSA

- SSA (Static Single Assignment) 是一种中间表示 (IR) 的形式, 其核心规则是: 每个变量在静态程序中被赋值且仅被赋值一次。

例 (非 SSA)

```
x = 1;  
x = x + 2;
```

例 (SSA)

```
x1 = 1;  
x2 = x1 + 2; // 每次赋值生成新变量名 (x1 → x2)
```

SSA

- SSA (Static Single Assignment) 是一种中间表示 (IR) 的形式, 其核心规则是: 每个变量在静态程序中被赋值且仅被赋值一次。
- 当程序存在控制流分支时, 使用 Phi 函数在分支合并点选择不同路径的值。

例 (Phi 函数)

```
if (cond) {  
    x1 = 1;  
} else {  
    x2 = 2;  
}  
x3 =  $\Phi$ (x1, x2); //  $\Phi$  函数根据路径选择 x1 或 x2  
y1 = x3 + 3;
```

SSA

- SSA (Static Single Assignment) 是一种中间表示 (IR) 的形式, 其核心规则是: 每个变量在静态程序中被赋值且仅被赋值一次。
- 当程序存在控制流分支时, 使用 Phi 函数在分支合并点选择不同路径的值。
- SSA 的意义:
 - 简化数据流分析
 - 变量仅一次赋值, 消除多次赋值带来的歧义。
 - 显式定义 - 使用链 (Def-Use Chain), 便于快速追踪依赖关系。

SSA

- SSA (Static Single Assignment) 是一种中间表示 (IR) 的形式, 其核心规则是: 每个变量在静态程序中被赋值且仅被赋值一次。
- 当程序存在控制流分支时, 使用 Phi 函数在分支合并点选择不同路径的值。
- SSA 的意义:
 - 简化数据流分析
 - 变量仅一次赋值, 消除多次赋值带来的歧义。
 - 显式定义 - 使用链 (Def-Use Chain), 便于快速追踪依赖关系。
 - 优化效率提升
 - 常量传播、公共子表达式消除、死代码删除等优化可直接在 SSA 上高效实现。例如: $x_2 = x_1 + 1$ 中若 x_1 是常量, 则 x_2 可直接替换为计算结果。

SSA

- SSA (Static Single Assignment) 是一种中间表示 (IR) 的形式, 其核心规则是: 每个变量在静态程序中被赋值且仅被赋值一次。
- 当程序存在控制流分支时, 使用 Phi 函数在分支合并点选择不同路径的值。
- SSA 的意义:
 - 简化数据流分析
 - 变量仅一次赋值, 消除多次赋值带来的歧义。
 - 显式定义 - 使用链 (Def-Use Chain), 便于快速追踪依赖关系。
 - 优化效率提升
 - 常量传播、公共子表达式消除、死代码删除等优化可直接在 SSA 上高效实现。
例如: $x_2 = x_1 + 1$ 中若 x_1 是常量, 则 x_2 可直接替换为计算结果。
 - 降低算法复杂度
 - 传统数据流分析需迭代计算, 而 SSA 通过 Φ 函数显式合并路径, 减少计算量。

SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。



SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。
- 变量重命名
 - 遍历支配树 (Dominator Tree), 为每个变量赋予唯一版本号。
例如: $x \rightarrow x1, x2, x3$ 。
 - 替换变量的引用为对应版本。

SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。
- 变量重命名
 - 遍历支配树 (Dominator Tree), 为每个变量赋予唯一版本号。
例如: $x \rightarrow x1, x2, x3$ 。
 - 替换变量的引用为对应版本。
- 处理非 SSA 场景
 - 内存操作: 如 $*p = 10$ 可能破坏 SSA, 需将其转换为显式加载 / 存储。
 - 函数调用: 保守假设所有全局变量可能被修改, 需插入 Φ 函数。

SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。
- 变量重命名
 - 遍历支配树 (Dominator Tree), 为每个变量赋予唯一版本号。
例如: $x \rightarrow x1, x2, x3$ 。
 - 替换变量的引用为对应版本。
- 处理非 SSA 场景
 - 内存操作: 如 $*p = 10$ 可能破坏 SSA, 需将其转换为显式加载 / 存储。
 - 函数调用: 保守假设所有全局变量可能被修改, 需插入 Φ 函数。
- 特殊处理方法:

SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。
- 变量重命名
 - 遍历支配树 (Dominator Tree), 为每个变量赋予唯一版本号。
例如: $x \rightarrow x1, x2, x3$ 。
 - 替换变量的引用为对应版本。
- 处理非 SSA 场景
 - 内存操作: 如 $*p = 10$ 可能破坏 SSA, 需将其转换为显式加载 / 存储。
 - 函数调用: 保守假设所有全局变量可能被修改, 需插入 Φ 函数。
- 特殊处理方法:
 - 惰性插入 Φ 函数 (仅在需要优化的变量上插入)。

SSA 的实现

- 插入 Φ 函数 - 支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块 (Basic Block) 的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Φ 函数。
- 变量重命名
 - 遍历支配树 (Dominator Tree), 为每个变量赋予唯一版本号。
例如: $x \rightarrow x1, x2, x3$ 。
 - 替换变量的引用为对应版本。
- 处理非 SSA 场景
 - 内存操作: 如 $*p = 10$ 可能破坏 SSA, 需将其转换为显式加载 / 存储。
 - 函数调用: 保守假设所有全局变量可能被修改, 需插入 Φ 函数。
- 特殊处理方法:
 - 惰性插入 Φ 函数 (仅在需要优化的变量上插入)。
 - 循环头基本块插入 Φ 函数, 用于处理循环体更新的值。

通过 Phi 函数选择的变量版本的数据保存到内存。
指令，将值传递到 Phi 函数的返回值位置。

- 方法一：使用内存操作，将需要通过 Phi 函数选择的变量版本的数据保存到内存对应的位置中；
- 方法二：在每个前驱块末尾插入指令，将值传递到 Phi 函数的返回值位置。

代数恒等式优化 (Algebraic Simplification)

- 利用数学规则简化表达式

例

优化前	优化后
$x + 0$	x
$x * 1$	x
$(a \ll 3) \ll 2$	$a \ll 5$
<code>int x = 2 * 2 * 2 * 2;</code>	<code>int x = 1 << 4;</code>

循环展开 (Loop Unrolling)

- 原理：减少循环控制语句的开销，增加指令级并行性。

例

优化前	优化后
<code>for (int i=0; i<4; i++) sum += a[i];</code>	<code>sum += a[0] + a[1] + a[2] + a[3];</code>

尾递归优化 (Tail Recursion Elimination)

- 原理：将尾递归转换为循环，避免栈溢出。同时减少栈帧管理所需要的额外指令。

例

优化前	优化后
<pre>int factorial(int n, int acc) { if (n == 0) return acc; return factorial(n-1, acc * n); }</pre>	<pre>int factorial(int n, int acc) { while (n > 0) acc *= n; n--; return acc; }</pre>